Fundamentos Técnicos de Backend, APIs e Arquitetura com .NET

Guia Pedagógico Completo: Do Navegador ao Banco de Dados com C#, ASP.NET Core e EF Core

Programação Web • Ciências de Dados para Negócios

PLANEJAMENTO PEDAGÓGICO

Roteiro da Aula

- Arquitetura Web, Cliente/Servidor, DNS, IP, Portas e Localhost

- HTTP em Profundidade: Request, Response, Métodos, Headers e Status

- Pausa e assimilação teórica

- APIs, REST, Formato JSON, Serialização e Operações CRUD

- Arquitetura Backend: Controller, Service, Repository, EF Core e Regras

- Desenho Arquitetural do DataSales e Rastreamento de um POST

BLOCO 1: FUNDAMENTOS

O Que É Backend de Verdade: Muito Além da Persistência

Definição Conceitual

- Backend NÃO é apenas 'a parte que salva dados no banco'.
- É a camada invisível responsável pelo processamento pesado, segurança e integridade.
- Orquestra a comunicação entre múltiplos clientes (Web, Mobile) e repositórios.
- Garante a execução consistente das Regras de Negócio da empresa.
- Fornece APIs padronizadas para consumir dados de forma segura.

Fluxo Básico de Execução

1. Cliente envia HTTP Request

2. Servidor Web recebe e filtra

3. Aplicação Backend executa Regras

4. Camada de Dados consulta o Banco

5. Resposta JSON é enviada ao Cliente

BLOCO 1: FUNDAMENTOS

Divisão Clara de Responsabilidades: Frontend vs. Backend

Responsabilidades do Frontend

- HTML: Estrutura semântica da interface do usuário.
- CSS: Estilização visual, leiaute e responsividade.
- JavaScript: Interatividade na página e captura de eventos.
- Experiência do Usuário (UX): Feedback imediato ao digitar.
- ⚠️ Importante: O usuário tem controle TOTAL do Frontend (DevTools, modificação de JS). Nenhuma segurança pode ser garantida apenas aqui!

Responsabilidades do Backend

- Validação final e rigorosa de dados recebidos.
- Autenticação e Autorização de segurança.
- Execução de Regras de Negócio e cálculos financeiros.
- Consulta e alteração com segurança no Banco de Dados.
- Auditoria, geração de logs e tratamento resiliente de erros.
- Integração com serviços de terceiros (Pagamentos, e-mails).

O Caminho de uma Requisição: DNS, IP e Criptografia TLS

Análise do Fluxo do Navegador até o Servidor

- 1. Digitação da URL: O usuário digita 'https://meusistema.com/vendas'.
- 2. Resolução DNS (Domain Name System): O navegador consulta servidores DNS para traduzir o nome de domínio para um Endereço IP numérico (ex: 203.0.113.10).
- 3. Handshake TLS/SSL (HTTPS): Estabelece um túnel criptografado seguro entre cliente e servidor.
- 4. Conexão TCP e Envio do Request: O pacote HTTP Request é transmitido para a porta do servidor.

Mapeamento Visual da Conexão Web:

```
meusistema.com  ——(DNS)——>  IP: 203.0.113.10  ——(Porta 443 HTTPS)——>  Servidor Web (Kestrel/.NET)
```

BLOCO 1: ARQUITETURA WEB

Anatomia Técnica de uma URL: Desconstruindo Endereços

`https://api.datasales.com:443/vendas/125?detalhes=true`

Parte da URL	Exemplo no Endereço	Função Técnica
Protocolo	`https`	Define as regras de comunicação e segurança (HTTP/HTTPS).
Host / Domínio	`api.datasales.com`	Identifica o servidor ou serviço na rede mundial.
Porta	`443`	Identifica o ponto final/processo (443 é o padrão para HTTPS; 80 para HTTP).
Path (Caminho)	`/vendas/125`	Mapeia o recurso específico no servidor (Entidade Vendas, ID 125).
Query String	`?detalhes=true`	Parâmetros opcionais para filtro, ordenação ou modificação da consulta.

BLOCO 1: ARQUITETURA WEB

Localhost, IP de Loopback e Isolamento de Processos

Conceito de Localhost e Loopback

- 'localhost' é o alias técnico para a própria máquina local.
- Mapeia para o IP de Loopback 127.0.0.1 (IPv4) ou ::1 (IPv6).
- Usado durante o desenvolvimento para testar o Backend antes de implantar na nuvem.
- A rede interna simula a comunicação de servidor real sem trafegar pela internet.

Portas de Rede e Múltiplos Processos

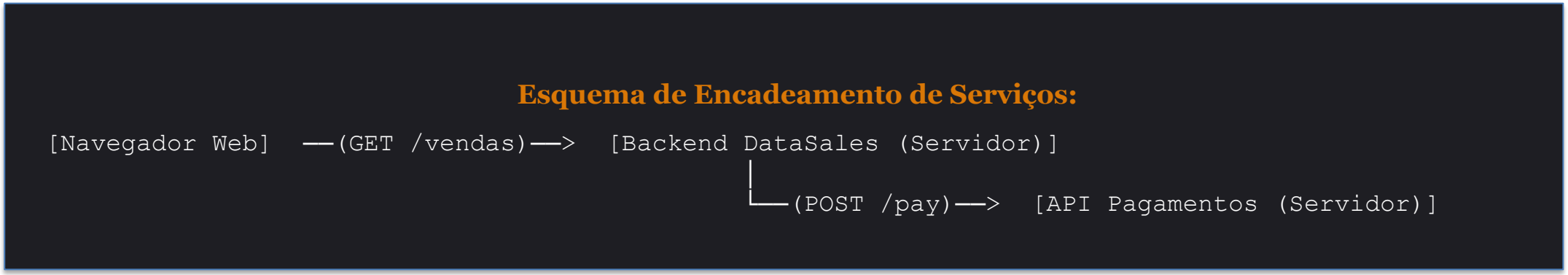
- Uma porta identifica um serviço ou aplicação específica executando na mesma máquina.
- Permite rodar várias aplicações simultaneamente sem conflito:
- • http://localhost:3000 ➔ Frontend React/Vue
- • http://localhost:5000 ➔ Backend API .NET
- • http://localhost:5432 ➔ Banco PostgreSQL
- 'Rodar o Backend' significa colocar o servidor escutando em uma porta dedicada.

Arquitetura Cliente-Servidor: Papéis Dinâmicos

Definição de Papéis Arquiteturais e Encadeamento

- Cliente: Qualquer entidade que inicia uma solicitação de recurso ou serviço.
- Servidor: Entidade que escuta, processa requisições e retorna uma resposta.
- Papéis Dinâmicos: Um sistema pode ser servidor em uma relação e cliente em outra!
- Exemplo de Encadeamento: O Frontend (Navegador) faz requisição ao Backend DataSales (Servidor). Por sua vez, o Backend DataSales consulta a API do Gateway de Pagamento (atuando como Cliente).

Esquema de Encadeamento de Serviços:



HTTP em Profundidade: O Ciclo Request-Response

Características do Protocolo HTTP

- Hypertext Transfer Protocol: O protocolo base da Web.
- Baseado em texto e orientado a mensagens.
- Modelo Request-Response: O cliente envia uma pergunta e aguarda a resposta do servidor.
- Stateless por padrão: Cada mensagem é independente.
- Padronizado pela W3C e IETF para garantir interoperabilidade universal.

As Três Etapas Indissociáveis

1. HTTP Request

Enviado pelo cliente contendo Método, URL, Headers e Body.

2. Processamento

Servidor valida, aplica regras de negócio e acessa o banco.

3. HTTP Response

Devolvido ao cliente contendo Status Code, Headers e Dados (JSON).

Estrutura Detalhada de uma HTTP Request

C# / .NET Snippet: HTTP Request Raw

```
POST /api/vendas HTTP/1.1
Host: api.datasales.com:443
Content-Type: application/json
Authorization: Bearer eyJhbGci...

{
  "produtoId": 15,
  "quantidade": 2,
  "precoUnitario": 3500.00
}
```

Componentes do Pacote

- Linha de Requisição: Método (POST), Path (/api/vendas) e Versão (HTTP/1.1).
- Host Header: Identifica o domínio alvo do servidor.
- Content-Type: Informa o formato dos dados enviados no Body (ex: application/json).
- Authorization Header: Transporta o token de segurança (Bearer Token).
- Body (Corpo): Carga útil de dados estruturados em JSON enviada ao servidor.

BLOCO 2: HTTP

Métodos HTTP e a Analogia com Operações CRUD e SQL

Operação CRUD	Verbo HTTP	Comando SQL	Significado na Aplicação
Create (Criar)	`POST`	`INSERT`	Cria um novo recurso ou executa um processo de cadastro.
Read (Ler)	`GET`	`SELECT`	Consulta e recupera informações sem alterar o estado do servidor.
Update (Atualizar)	`PUT / PATCH`	`UPDATE`	Atualiza um recurso existente (Substituição total ou parcial).
Delete (Excluir)	`DELETE`	`DELETE`	Remove um recurso específico do sistema.

⚠️ Princípio Fundamental: HTTP Não É SQL!

Um endpoint HTTP representa um RECURSO ou CASO DE USO da aplicação, e não um comando de banco de dados direto. O Backend decide internamente como processar a requisição.

BLOCO 2: HTTP

Diferença Crucial de Atualização: PUT vs. PATCH

PUT: Substituição Integral

- Usado para substituir COMPLETAMENTE o recurso.
- O cliente deve enviar todos os campos do objeto.
- Se um campo opcional for omitido, ele pode ser sobrescrito como nulo no servidor.

C# / .NET Snippet: Payload PUT

```
PUT /usuarios/15
{
  "nome": "Brenda",
  "email": "brenda@email.com",
  "cidade": "Sorocaba"
}
```

PATCH: Modificação Parcial

- Usado para modificar APENAS os campos informados.
- Ideal para economizar banda e atualizar atributos específicos (ex: apenas a cidade).
- Preserva todos os outros dados existentes no recurso.

C# / .NET Snippet: Payload PATCH

```
PATCH /usuarios/15
{
  "cidade": "Campinas"
}
```

Headers HTTP: Metadados Cruciais de Comunicação

A Função dos Metadados em Requests e Responses

- Headers são pares chave-valor que transportam metadados essenciais sobre a requisição ou resposta.
- Não fazem parte do corpo dos dados (body), mas definem como o corpo deve ser interpretado.
- Essenciais para controle de segurança, negociação de conteúdo e controle de cache.

Content-Type

- Informa ao servidor o formato do dado enviado no Body.

```
Content-Type: application/json
```

Authorization

- Transporta credenciais de autenticação/tokens.

```
Authorization: Bearer eyJhbGci...
```

Accept

- Informa ao servidor qual formato o cliente aceita receber.

```
Accept: application/json
```

Path Parameters vs. Query Parameters

Path Parameters (Identificação)

- Fazem parte da estrutura fixa da URL.
- Usados para identificar um RECURSO ESPECÍFICO e único.
- Exemplo: GET /clientes/125
- O valor '125' identifica o cliente de ID 125.

C# / .NET Snippet: Path Param Example

```
GET /api/vendas/584/itens/2
// Identifica a Venda 584, Item 2
```

Query Parameters (Filtro e Opções)

- Iniciados por '?' e separados por '&'.
- Usados para FILTROS, PAGINAÇÃO, ORDENAÇÃO e buscas.
- Exemplo: GET /vendas?ano=2026&pagina=2
- Não alteram a identidade do recurso, apenas como ele é apresentado.

C# / .NET Snippet: Query Param Example

```
GET /api/vendas?inicio=2026-01-01&status=paga
// Filtra vendas pagas em 2026
```

O Conceito de API: Contrato de Comunicação

O Que É uma API?

- API = Application Programming Interface.
- Interface padronizada que permite a comunicação entre dois softwares distintos.
- Funciona como um 'Contrato de Comunicação': define regras, métodos e formatos aceitos.
- Desacopla o Frontend do Backend: o cliente não precisa saber qual linguagem ou banco o servidor utiliza.

Contrato de Endpoints da API

C# / .NET Snippet: Contrato de Recursos HTTP

```
GET    /api/produtos     ➔ Listar todos os produtos
GET    /api/produtos/15  ➔ Obter produto ID 15
POST   /api/produtos     ➔ Criar novo produto
PUT    /api/produtos/15  ➔ Atualizar produto ID 15
DELETE /api/produtos/15  ➔ Remover produto ID 15
```

Estilo Arquitetural REST: Orientação a Recursos

Abordagem Legada (RPC)

- Focada em 'Ações' e 'Funções' na URL.
- Multiplicação desordenada de URLs no servidor.
- Dificulta a padronização e manutenção da API.

C# / .NET Snippet: Estilo RPC (Orientado a Ações)

```
/getClientes
/criarCliente
/deletarCliente?id=10
/atualizarCliente
```

Abordagem RESTful (Recursos)

- Focada em RECURSOS (Substantivos no plural).
- A ação é definida pelo Verbo HTTP (GET, POST, PUT, DELETE).
- URLs limpas, intuitivas e padronizadas universalmente.

C# / .NET Snippet: Estilo REST (Orientado a Recursos)

```
GET    /clientes     → Listar
POST   /clientes     → Criar
GET    /clientes/10  → Obter 10
DELETE /clientes/10  → Deletar 10
```

O Formato JSON Tecnicamente: Sintaxe e Tipos

C# / .NET Snippet: JSON Estruturado Exemplo

```
{
  "vendaId": 584,
  "cliente": {
    "id": 42,
    "nome": "Maria Silva"
  },
  "itens": [
    {"produto": "Notebook", "qtd": 1, "preco": 3500.00},
    {"produto": "Mouse", "qtd": 2, "preco": 100.00}
  ],
  "valorTotal": 3700.00,
  "paga": true
}
```

Sintaxe e Tipos Primitivos

- JSON = JavaScript Object Notation.
- Formato TEXTUAL de intercâmbio de dados leve e legível por humanos.
- Tipos de Dados Suportados:
- • String: "Maria Silva"
- • Number: 584, 3500.00
- • Boolean: true / false
- • Null: null
- • Object: { "id": 42 } (Chave-Valor)
- • Array: [...] (Lista ordenada de elementos)

JSON vs. Banco de Dados: Esclarecendo o Papel

JSON: Formato de Transporte

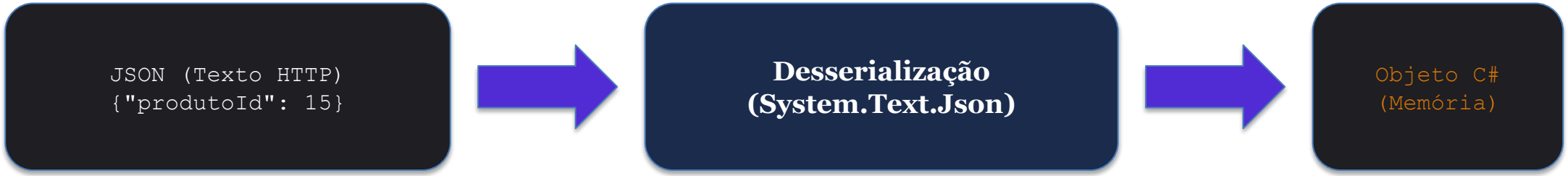
- JSON é uma REPRESENTAÇÃO TEXTUAL de dados em trânsito.
- Usado para enviar e receber informações pela rede via HTTP.
- NÃO possui índice de busca rápida, transações ACID ou persistência segura em disco por si só.
- É efêmero: existe na memória do cliente e na requisição.

Banco de Dados: Camada de Persistência

- Mecanismo especializado em ARMAZENAMENTO PERMANENTE e estruturado.
- Garante integridade referencial, relacionamentos entre tabelas e índices.
- Suporta concorrência de milhares de acessos simultâneos.
- Conclusão: O JSON é o envelope de transporte; o Banco de Dados é o cofre de armazenamento!

Serialização e Desserialização de Objetos C#

O Ciclo de Conversão de Dados na API .NET



- Desserialização (Entrada): O ASP.NET Core pega o texto JSON recebido na HTTP Request e o converte automaticamente em uma instância de objeto C# (DTO).
- Processamento Interno: A aplicação manipula fortemente tipos C# fortemente tipados na memória.
- Serialização (Saída): Ao retornar um resultado na Controller, o framework converte o objeto C# de volta em texto JSON para trafegar no HTTP Response.

BLOCO 4: ARQUITETURA BACKEND

Arquitetura Backend: Do 'Código Espaguete' às Camadas

❌ Backend Sem Organização

- Tudo concentrado no mesmo arquivo/função:
- Recebe HTTP ➔ Valida Usuário ➔ Executa SQL ➔ Calcula Desconto ➔ Envia E-mail ➔ Atualiza Estoque ➔ Monta JSON.
- Consequências Desastrosas:
- • Impossível de testar em separado.
- • Código duplicado em vários pontos.
- • Alto risco de quebrar o sistema a cada alteração.
- • Violação total do Princípio da Responsabilidade Única (SRP).

✅ Backend Organizado em Camadas

- Separação Clara de Responsabilidades:
- • Controller ➔ Trata a fronteira HTTP.
- • Service ➔ Modela as Regras de Negócio.
- • Repository ➔ Isola o acesso ao Banco.
- Benefícios Diretamente Mensuráveis:
- • Facilidade de manutenção e evolução.
- • Testabilidade automatizada simples.
- • Baixo acoplamento entre componentes.

BLOCO 4: ARQUITETURA BACKEND

Visão Geral: Controller ➔ Service ➔ Repository

A Triade Fundamental do Backend .NET

1. Controller

FRONTEIRA HTTP

- Recebe a requisição HTTP, extrai os parâmetros do DTO, chama a camada de serviço e retorna a resposta com o Status Code adequado.

2. Service

REGRAS DE NEGÓCIO

- Coração da aplicação. Valida condições de negócio, aplica descontos, verifica estoque e orquestra casos de uso puramente em C#.

3. Repository

ACESSO AOS DADOS

- Abstrai a tecnologia do banco de dados (EF Core / SQL). Executa os métodos de consulta (SELECT) e persistência (INSERT/UPDATE).

Validação de Dados: Frontend vs. Backend

Validação no Frontend (UX)

- Exemplo: `<input type='number' min='1'>` no HTML.
- Objetivo Principal: Melhora a Experiência do Usuário (UX) fornecendo feedback visual imediato.
- Evita requisições desnecessárias ao servidor para erros simples de digitação.
- ⚠️ Vulnerabilidade: Pode ser burla facilmente com F12 / DevTools ou Postman!

Validação no Backend (Segurança)

- Exemplo no .NET: Rejeitar `quantidade <= 0` com `400 Bad Request`.
- Objetivo Principal: Proteção e Integridade Absoluta da Aplicação e do Banco.
- Garante que dados maliciosos ou corrompidos JAMAIS sejam processados.
- Regra de Ouro: Validação de Frontend é opcional para o servidor; a de Backend é OBRIGATÓRIA!

BLOCO 5: RESILIÊNCIA & SEGURANÇA

Famílias de Status Codes HTTP

1xx — Informacional
A requisição foi recebida e o processo continua.

2xx — Sucesso
A ação solicitada foi recebida, compreendida e aceita com êxito.

3xx — Redirecionamento
Ações adicionais são necessárias para concluir a requisição.

4xx — Erro do Cliente
A requisição contém sintaxe incorreta ou não pode ser processada pelo cliente.

5xx — Erro do Servidor
O servidor falhou ao processar uma requisição aparentemente válida.

BLOCO 5: RESILIÊNCIA & SEGURANÇA

Tabela dos Status Codes Fundamentais na API

Código	Nome / Significado	Situação Típica de Uso na API
200 OK	Sucesso Genérico	Retorno bem-sucedido de consultas (GET) ou edições.
201 Created	Recurso Criado	Sucesso na criação de um novo registro (POST /vendas).
204 No Content	Sucesso Sem Corpo	Ação concluída com sucesso sem dados de retorno (DELETE).
400 Bad Request	Requisição Inválida	Dados de entrada incorretos ou falha nas regras de validação.
401 Unauthorized	Não Autenticado	Credenciais de login ausentes ou token expirado.
403 Forbidden	Sem Permissão	Usuário autenticado, mas sem privilégio para essa ação.
404 Not Found	Não Encontrado	O recurso ou ID solicitado não existe no banco.
500 Internal Error	Erro de Servidor	Exceção não tratada ou falha crítica de infraestrutura.

Autenticação vs. Autorização no Backend

Autenticação: 'Quem é você?'

- Processo de VERIFICAÇÃO DE IDENTIDADE do usuário.
- Exemplo: Login com e-mail e senha, biometria ou MFA.
- Se bem-sucedido, o servidor gera uma credencial/token assinado comprovando a identidade.

C# / .NET Snippet: Login Endpoint

```
POST /api/login
{
  "email": "joao@email.com",
  "senha": "123456"
}
➔ Retorna Token JWT
```

Autorização: 'O que você pode fazer?'

- Processo de VERIFICAÇÃO DE PERMISSÕES e privilégios.
- Exemplo: O usuário 'João' (Autenticado) pode deletar uma venda?
- Valida se o perfil do usuário (Admin, Gerente, Operador) permite executar o endpoint.

C# / .NET Snippet: Check de Permissão

```
DELETE /api/vendas/584
Headers:
  Authorization: Bearer TokenJoão
➔ Retorna 403 Forbidden (se não for Admin)
```

BLOCO 5: RESILIÊNCIA & SEGURANÇA

Tokens de Acesso e o Padrão Bearer Token

O Fluxo de Autenticação Baseado em Token (JWT)



- Header de Autorização: Nas requisições subsequentes, o cliente envia o token no Header:
- `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`
- Assinatura Criptográfica: O servidor valida o token sem precisar consultar o banco de dados a cada requisição.

BLOCO 5: RESILIÊNCIA & SEGURANÇA

Comunicação Stateless e Escalabilidade

O Conceito de Stateless

- Cada requisição é COMPLETA e INDEPENDENTE.
- O servidor não guarda estado da sessão do usuário na memória local.
- Toda requisição deve carregar todas as informações necessárias para ser processada (ex: Token de autenticação).
- Inexistência de 'memória conversacional' entre requisições consecutivas.

A Grande Vantagem: Escalabilidade Horizontal

- Permite distribuir as requisições entre 10 ou 100 servidores Backend iguais (Load Balancer).
- Como nenhum servidor guarda sessão local, qualquer servidor pode responder à Requisição 2 do mesmo usuário!
- Garante alta disponibilidade e capacidade de atender milhões de acessos simultâneos sem gargalos.

BLOCO 5: RESILIÊNCIA & SEGURANÇA

Tratamento de Erros e Exceções no Server

Fluxo de Captura, Sanitização e Resposta

- Um Backend profissional NUNCA deve expor detalhes de infraestrutura ou Stack Traces ao cliente.
- Falhas de entrada ➔ Devolver `400 Bad Request` com lista clara de erros de validação.
- Recurso inexistente ➔ Devolver `404 Not Found`.
- Erros inesperados de servidor ➔ Devolver `500 Internal Server Error` com mensagem genérica amigável ao usuário, enquanto registra o erro detalhado nos logs do servidor.

C# / .NET Snippet: Resposta de Erro Padronizada (ProblemDetails)

```
{
  "status": 400,
  "erro": "RequisicaoInvalida",
  "detalhes": [
    "A quantidade deve ser maior que zero.",
    "O produto com ID 999 nao foi encontrado."
  ]
}
```

BLOCO 5: RESILIÊNCIA & SEGURANÇA

Logs Estruturados e Observabilidade em Produção

A Importância dos Logs Estruturados

- Sem logs, o Backend é uma caixa preta inacessível em produção.
- Logs registraram eventos críticos: Início de requisição, erros, consultas lentas e acessos negados.
- Logs Estruturados (JSON): Facilitam a indexação e busca rápida em ferramentas como ElasticSearch, Datadog ou Serilog.

C# / .NET Snippet: Log de Produção Exemplo

```
2026-09-29 14:30:12 [INFO] POST /api/vendas
  UsuarioId: 125
  Status: 201 Created
  TempoProcessamento: 184ms
  DatabaseTime: 42ms
```

BLOCO 6: ECOSSISTEMA .NET

O Ecossistema .NET Desmistificado

Tecnologia	O Que É de Verdade?	Função no Ecossistema
C# (C-Sharp)	Linguagem de Programação	Linguagem moderna, orientada a objetos e fortemente tipada usada para escrever o código.
.NET	Plataforma de Desenvolvimento	Ecossistema completo de execução (CLR), bibliotecas base e compilador.
ASP.NET Core	Framework Web	Framework de alta performance para construir APIs REST e aplicações Web no .NET.
Entity Framework Core	ORM (Object-Relational Mapper)	Ferramenta que mapeia classes C# diretamente para tabelas do banco SQL.

ASP.NET Core: Controllers, Atributos e Binding

C# / .NET Snippet: Controller e Atributos .NET

```
[ApiController]
[Route("api/[controller]")] // Rota: api/vendas
public class VendasController : ControllerBase
{
    [HttpGet("{id}")] // Path param: api/vendas/125
    public IActionResult ObterPorId(int id)
    {
        return Ok(new { id = id, prod = "Notebook" });
    }
}
```

Recursos do ASP.NET Core

- `[ApiController]`: Habilita comportamentos automáticos de API (validação automática de Model State).
- `[Route]`: Define o padrão de URL para a controller.
- Model Binding Automático: O ASP.NET Core converte JSON, Query Params e Path Params diretamente em parâmetros dos métodos C#.

Entity Framework Core (EF Core): O ORM do .NET

Mapeamento Objeto-Relacional

C# / .NET Snippet: LINQ x SQL com EF Core

```
// Consulta fortemente tipada em C# (LINQ)
var vendas = _context.Vendas
    .Where(v => v.ValorTotal > 1000)
    .ToList();

// O EF Core traduz para SQL automaticamente:
// SELECT * FROM Vendas WHERE ValorTotal > 1000
```

Vantagens do EF Core

- Elimina a necessidade de escrever SQL string concatenado manualmente.
- Proteção automática contra SQL Injection.
- Migrações de Banco de Dados (Migrations): Controle de versão da estrutura do banco via código C#.
- Aumento expressivo de produtividade no desenvolvimento.

BLOCO 6: ECOSSISTEMA .NET

DTOs (Data Transfer Objects) vs. Entidades de Domínio

Característica	Entidade do Banco (Entity)	Objeto de Transferência (DTO)
Objetivo Principal	Representa a estrutura de persistência no Banco.	Modela estritamente o contrato público da API.
Exposição de Dados	Contém todos os campos internos (inclusive dados sensíveis).	Expõe APENAS os campos necessários para a operação.
Uso no Sistema	Utilizada internamente nas camadas Service e Repository.	Utilizada nas bordas da aplicação (Requests / Responses).
Exemplo de Campo	SenhaHash, DataCriacao, ChaveInterna.	Apenas Email, Nome, Quantidade.

BLOCO 6: ECOSSISTEMA .NET

Injeção de Dependência (DI) e Ciclos de Vida

Inversão de Controle (IoC)

- Sem DI: A Controller instancia manualmente `new VendaService()`.
- Com DI: A Controller DECLARA que precisa de `IVendaService` no seu construtor, e o ASP.NET Core injeta a instância pronta automaticamente!
- Elimina o acoplamento rígido entre classes e facilita a substituição de componentes.

Os 3 Ciclos de Vida no .NET

- `AddScoped`: Uma nova instância é criada por REQUISIÇÃO HTTP (Ideal para Services, Repositories e DbContext).
- `AddTransient`: Uma nova instância é criada TODA VEZ que a dependência é solicitada (Objetos leves sem estado).
- `AddSingleton`: Uma ÚNICA instância é criada e compartilhada por toda a aplicação (Caches, Configurações globais).

Interfaces e Baixo Acoplamento na Prática

C# / .NET Snippet: IVendaRepository.cs

```
public interface IVendaRepository
{
    void Salvar(Venda venda);
    Venda ObterPorId(int id);
}

// Implementação Real (SQL Server)
public class VendaRepositorySql : IVendaRepository { ... }

// Implementação MOCK para Testes Unitários
public class VendaRepositoryMock : IVendaRepository { ... }
```

Por Que Usar Interfaces?

- O Service depende do CONTRATO (`IVendaRepository`), e não da implementação concreta.
- Permite trocar a tecnologia do banco sem alterar uma única linha de código da Service.
- Permite rodar Testes Unitários ultrarrápidos utilizando repositórios falsos em memória (Mocks).

BLOCO 7: VISÃO ENTERPRISE

Arquitetura Enterprise em 4 Camadas

A Organização de Sistemas Corporativos Robustos

1. Presentation Layer

Controllers, Endpoints, Requests, Responses, Middlewares e Swagger.

2. Application Layer

Casos de Uso, Mapeamento de DTOs e Orquestração de Serviços.

3. Domain Layer

Entidades de Negócio, Validações de Domínio e Regras Fundamentais.

4. Infrastructure Layer

Entity Framework Core, Repositories, SQL Server, APIs Externas e Logs.

BLOCO 7: VISÃO ENTERPRISE

Monolito Organizado vs. Microserviços: Projeto DataSales

Monolito Modular (Escolha Adequada)

- Toda a aplicação DataSales é desenvolvida e implantada como uma única unidade.
- Vantagens para o Projeto Didático:
- • Simplicidade de implantação e depuração.
- • Sem complexidade de rede ou falhas distribuídas.
- • Ideal para equipes iniciais e sistemas em fase de consolidação.
- ⚠️ Monolito organizado em camadas NÃO é código ruim!

Microserviços (Complexidade Distribuída)

- Serviços independentes (Clientes, Vendas, Pagamentos) com bancos de dados próprios.
- Desafios Adicionados:
- • Latência de rede entre chamadas de serviços.
- • Necessidade de mensageria (RabbitMQ / Kafka) e consistência eventual.
- • Alta complexidade de observabilidade e deploy.
- Conclusão: Deve ser adotado por necessidade de escala de times, não por moda!

BLOCO 7: VISÃO ENTERPRISE

Middlewares e o Pipeline HTTP do ASP.NET Core

C# / .NET Snippet: Program.cs Pipeline

```
// Program.cs - Pipeline de Execução Sequencial
var app = builder.Build();

app.UseExceptionHandler("/error"); // 1. Erros Globais
app.UseHttpsRedirection();         // 2. HTTPS Redirection
app.UseAuthentication();           // 3. Autenticação
app.UseAuthorization();            // 4. Autorização
app.MapControllers();              // 5. Roteamento
```

Como Funciona o Pipeline?

- Uma requisição passa sequencialmente por cada Middleware antes de chegar na Controller.
- A ORDEM É CRUCIAL: A autenticação DEVE vir antes da autorização!
- Cada middleware pode interromper a requisição (ex: rejeitar requisição sem token antes de gastar recursos).

FECHAMENTO DIDÁTICO

Passo a Passo de um POST e Ponte para a Prática

Rastreamento do Clique ao Banco

1. Clique no botão HTML ➔ Envio do POST JSON

2. Pipeline de Middlewares ➔ Filtra e Autentica

3. VendasController ➔ Recebe CriarVendaRequest

4. VendaService ➔ Valida estoque e calcula total

5. VendaRepository ➔ Executa INSERT via EF Core

6. Retorno HTTP 201 Created com JSON gerado

Ponte para as Próximas Aulas

- Com este modelo mental consolidado, o aluno está pronto para a prática:
- 1. Subir uma aplicação Web API ASP.NET Core via SDK.
- 2. Criar os primeiros endpoints no VendasController.
- 3. Testar envios de JSON via Swagger / Postman.
- 4. Implementar gradualmente a separação em Service e Repository.
- 5. Conectar o Entity Framework Core ao banco SQL Server real!